

Vulnerability Triage Report

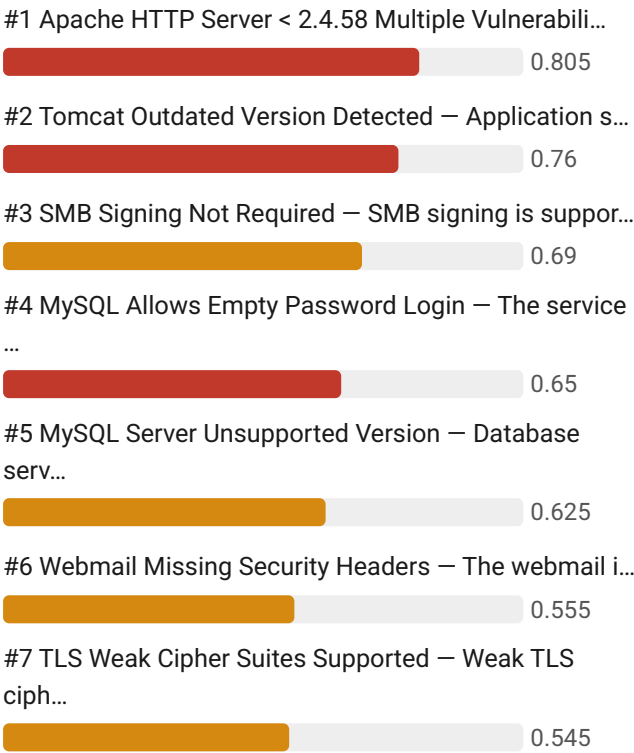
Generated by vulntrriage · 2026-07-09 21:42

Executive Summary

This report covers 14 vulnerability finding(s) triaged by the LLM-driven pipeline. Of these, 3 are rated High exploitability, 8 Medium, and 3 Low. Exploitability was scored by a 3-model ensemble (strict-majority quorum 2); 0 finding(s) were Unresolved (no label reached the quorum) and flagged for review. The highest-priority finding is: Apache HTTP Server < 2.4.58 Multiple Vulnerabilities – Outdated Apache version detected – Result: Banner returned: Apache/2.4.52 (Ubuntu) (host 192.168.10.25, risk score 0.805). Findings are ranked by a composite risk score combining CVSS, LLM-assessed exploitability, and asset criticality. Recommended remediation steps follow each finding.



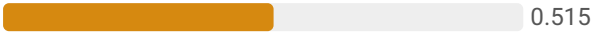
Risk Breakdown



#8 RDP Supports Weak Encryption – Remote Desktop se...



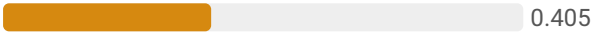
#9 SSH Weak Key Exchange Algorithms Supported – Wea...



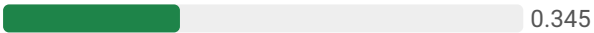
#10 SMTP Open Relay Check – Mail server configuratio...



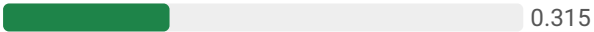
#11 Directory Listing Enabled – The web server appea...



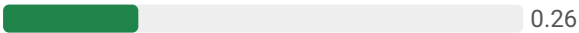
#12 TLS Certificate Expires Soon – TLS certificate i...



#13 HTTP TRACE Method Enabled – The HTTP TRACE metho...



#14 HTTP Server Header Disclosure – Server version d...



Technical Findings

#1 High Apache HTTP Server < 2.4.58 Multiple Vulnerabilities — Outdated Apache version detected — Result: Banner returned: Apache/2.4.52 (Ubuntu)

Host: 192.168.10.25:80

CVE: CVE-2026-10001

CVSS: 8.1

Risk score: 0.805 (asset criticality 0.5)

Context: The Apache HTTP Server version 2.4.52 is vulnerable to CVE-2026-10001, a critical flaw that enables remote code execution or denial of service via crafted HTTP requests. Attackers actively scan for outdated Apache installations to exploit this vulnerability, gaining initial access to the server and potentially pivoting to internal networks. Successful exploitation can result in full system compromise, theft of sensitive data, or service disruption. The business impact includes reputational damage, financial loss from downtime, and potential regulatory penalties for data breaches.

Exploitability rationale: 2/3 models: High=2, Medium=1 (quorum 2 -> High)

Ensemble votes: deepseek-v4-pro=High, glm-5.2=High, kimi-k2.6=Medium

Remediation: Upgrade Apache HTTP Server to version 2.4.58 or later to remediate multiple vulnerabilities including CVE-2026-10001.

1. Update the package list: `sudo apt update`
2. Upgrade the apache2 package: `sudo apt install --only-upgrade apache2`
3. Verify the installed version: `apache2 -v`
4. If 2.4.58 or newer is not available from the default repositories, add the Ubuntu security or backports repository (e.g., `sudo add-apt-repository 'deb http://archive.ubuntu.com/ubuntu $(lsb_release -sc)-security main'`) and repeat the upgrade, or consider upgrading the Ubuntu release to a supported version that ships a compliant Apache package.
5. Restart the Apache service: `sudo systemctl restart apache2`
6. Confirm the service is running and the banner no longer indicates an outdated version.

#2 High Tomcat Outdated Version Detected – Application server version is outdated – Result: Banner indicates Apache Tomcat/9.0.30

Host: 192.168.10.60:8443

CVSS: 7.2

Risk score: 0.76 (asset criticality 0.5)

Context: The host is running an outdated version of Apache Tomcat (9.0.30) that is missing critical security patches, leaving it susceptible to multiple known remote code execution and information disclosure exploits. Attackers can leverage publicly available exploits for vulnerabilities such as Ghostcat (CVE-2020-1938) to read arbitrary files or achieve remote code execution, potentially taking full control of the server. Successful exploitation could result in unauthorized access to sensitive business data, service disruption, and lateral movement within the internal network, leading to severe financial and reputational damage.

Exploitability rationale: 2/3 models: High=2, Medium=1 (quorum 2 -> High)

Ensemble votes: deepseek-v4-pro=High, glm-5.2=High, kimi-k2.6=Medium

Remediation: Upgrade to the latest stable Apache Tomcat release to address multiple known vulnerabilities and apply vendor security fixes.

1. Review Apache Tomcat 9.x security advisories and changelogs to identify all vulnerabilities and improvements released after version 9.0.30.
2. Create a full backup of the existing Tomcat installation directory, configuration files, custom libraries, and deployed web applications.
3. Download the latest stable Apache Tomcat 9.x release (or a supported major version if 9.x is end-of-life) from the official Apache website and verify its integrity with checksums or PGP signatures.
4. Set up a staging environment mirroring production, install the new version, and thoroughly test all applications for compatibility and regression.
5. Schedule a maintenance window, stop the Tomcat service, and replace the old installation with the new version, carefully merging custom configurations per the migration guide.
6. Start the Tomcat service, monitor logs for errors, and conduct functional and security testing to confirm proper operation.
7. Optionally, hide the Tomcat server version in HTTP headers (e.g., using the `ErrorReportValve`) to minimize information leakage.
8. Establish a regular patch management cycle and subscribe to the Apache Tomcat announce mailing list for future update notifications.

#3 Medium SMB Signing Not Required – SMB signing is supported but not required – Result: Message signing: enabled but not required

Host: 192.168.10.40:445

CVSS: 8.8

Risk score: 0.69 (asset criticality 0.5)

Context: The Windows SMB service does not enforce message signing, leaving it vulnerable to SMB relay and man-in-the-middle attacks. An attacker on the same network segment can intercept NTLM authentication handshakes and relay them to other systems to gain unauthorized access, potentially achieving remote code execution. This weakness is often exploited in Active Directory environments to move laterally and escalate privileges. Successful exploitation can lead to full domain compromise, sensitive data theft, and significant operational disruption.

Exploitability rationale: 2/3 models: High=1, Medium=2 (quorum 2 -> Medium)

Ensemble votes: deepseek-v4-pro=High, glm-5.2=Medium, kimi-k2.6=Medium

Remediation: Configure the server to require SMB signing to prevent man-in-the-middle relay attacks.

1. Open an elevated PowerShell or command prompt.
2. Set the registry value RequireSecuritySignature to 1 by running: `reg add HKLM\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters /v RequireSecuritySignature /t REG_DWORD /d 1 /f`
3. Ensure EnableSecuritySignature is also set to 1: `reg add HKLM\SYSTEM\CurrentControlSet\Services\LanmanServer\Parameters /v EnableSecuritySignature /t REG_DWORD /d 1 /f`
4. Restart the Server service to apply changes: `Restart-Service LanmanServer` (plan a maintenance window as SMB connections will be disrupted).
5. Verify the setting by running: `Get-SmbServerConfiguration | Select RequireSecuritySignature`

#4 High MySQL Allows Empty Password Login – The service accepted authentication with a blank password – Result: Anonymous or blank-password login succeeded

Host: 192.168.10.30:3306

CVSS: 5.0

Risk score: 0.65 (asset criticality 0.5)

Context: The MySQL server on port 3306 accepts authentication with an empty password, allowing unauthorized access to the database. Attackers can easily discover and exploit this misconfiguration by brute-forcing default or common usernames with a blank password, then read, modify, or exfiltrate sensitive data, and potentially execute operating system commands if the MySQL user has sufficient privileges. This can lead to full database compromise, data breaches, compliance violations, and severe business disruption.

Exploitability rationale: 2/3 models: High=2, Medium=1 (quorum 2 -> High)

Ensemble votes: deepseek-v4-pro=High, glm-5.2=Medium, kimi-k2.6=High

Remediation: Eliminate blank password authentication by setting strong passwords for all MySQL user accounts, especially root, and removing anonymous accounts.

1. Connect to MySQL as a privileged user (e.g., `mysql -u root` if the root password is blank, or `mysql -u root -p` and enter the known root password).
2. Execute the following query to list all users with empty passwords: `SELECT User, Host FROM mysql.user WHERE authentication_string = "" AND plugin = 'mysql_native_password';` (for MySQL 5.7+). For earlier versions, use `SELECT User, Host FROM mysql.user WHERE Password = "";`
3. For each account returned, set a strong password using: `ALTER USER '<user>'@'<host>' IDENTIFIED BY '<strong_password>';` (MySQL 5.7+) or `SET PASSWORD FOR '<user>'@'<host>' = PASSWORD('<strong_password>');` (older versions).
4. If the root account has a blank password, set it with: `ALTER USER 'root'@'localhost' IDENTIFIED BY '<strong_password>';` (adjust host as needed).
5. Remove anonymous accounts: `DROP USER "<user>@'localhost'"; DROP USER "<user>@";` (and any other hosts).
6. Run `FLUSH PRIVILEGES;` to apply changes.
7. Optionally, run `mysql_secure_installation` to automate these steps and apply additional hardening.
8. Verify the fix by attempting a blank password login: `mysql -u root`. The connection should be rejected.

#5 Medium MySQL Server Unsupported Version – Database server appears to be running an unsupported version – Result: Version fingerprint indicates MySQL 5.6.x

Host: 192.168.10.30:3306

CVSS: 7.5

Risk score: 0.625 (asset criticality 0.5)

Context: The MySQL 5.6 database server on 192.168.10.30:3306 is running an unsupported version that no longer receives security patches, leaving it susceptible to known and unknown vulnerabilities. In real-world attacks, threat actors can exploit unpatched flaws to compromise the database, steal sensitive data, or use it as a foothold for lateral movement within the network. The business impact includes potential data breaches, regulatory non-compliance penalties, and disruption of critical services reliant on the database.

Exploitability rationale: 3/3 models: Medium=3 (quorum 2 -> Medium)

Ensemble votes: deepseek-v4-pro=Medium, glm-5.2=Medium, kimi-k2.6=Medium

Remediation: Upgrade MySQL 5.6 to a supported release (e.g., 8.0 or later) to mitigate risks from unpatched vulnerabilities and loss of vendor support.

1. Confirm the current MySQL version and take a full backup of all databases, configurations, and user accounts.
2. Review the MySQL upgrade documentation and application compatibility notes; test the upgrade on a staging server.
3. Schedule a maintenance window and notify stakeholders.
4. Stop the MySQL service, install the target MySQL version (e.g., 8.0), and run the `mysql_upgrade` utility.
5. Start the new MySQL service, verify data integrity, and re-test application connectivity and functionality.
6. Implement a regular patching cycle and monitor for new major releases to avoid future unsupported versions.

#6 Medium Webmail Missing Security Headers – The webmail interface is missing recommended HTTP security headers – Result: Missing: Content-Security-Policy, X-Frame-Options

Host: 192.168.10.50:443

CVSS: 6.1

Risk score: 0.555 (asset criticality 0.5)

Context: The webmail interface on host 192.168.10.50 is missing critical HTTP security headers—Content-Security-Policy (CSP) and X-Frame-Options. This deficiency leaves users vulnerable to clickjacking attacks, where a malicious site could transparently load the webmail login or actions in an iframe to trick victims into unknowingly performing sensitive operations. Without CSP, the application also lacks a robust defense against cross-site scripting (XSS), potentially allowing attackers to inject scripts to steal session cookies or redirect users to phishing pages. Successful exploitation could result in compromised email accounts, data breaches, and significant reputational and compliance damage to the organization.

Exploitability rationale: 2/3 models: Low=1, Medium=2 (quorum 2 -> Medium)

Ensemble votes: deepseek-v4-pro=Medium, glm-5.2=Medium, kimi-k2.6=Low

Remediation: Mitigate clickjacking and cross-site scripting risks by adding the missing X-Frame-Options and Content-Security-Policy HTTP security headers.

1. Determine the web server software (e.g., Apache, Nginx) and locate the configuration file for the webmail virtual host.
2. Add the X-Frame-Options header with value 'DENY' (or 'SAMEORIGIN' if framing by the same origin is required) to prevent clickjacking. For Apache: 'Header always set X-Frame-Options "DENY"'. For Nginx: 'add_header X-Frame-Options "DENY" always;'.
3. Implement a Content-Security-Policy header. Start with a restrictive baseline, such as 'default-src \'self\'; frame-ancestors \'self\';' and use report-only mode ('Content-Security-Policy-Report-Only') initially to avoid breaking functionality.
4. Reload or restart the web server to apply the new configuration.
5. Test the headers using browser developer tools (Network tab) or a command-line tool (e.g., 'curl -I https://webmail.example.com') to confirm the headers are present.
6. Monitor application behavior and refine the Content-Security-Policy to allow necessary external resources (e.g., images, fonts, scripts from email content) while maintaining a strong security posture. Switch from report-only to enforcing mode after validation.
7. Regularly review and update the security headers as the webmail application evolves or as new best practices emerge.

#7 Medium TLS Weak Cipher Suites Supported – Weak TLS cipher suites accepted – Result: TLS_RSA_WITH_3DES_EDE_CBC_SHA accepted

Host: 192.168.10.25:443

CVSS: 5.9

Risk score: 0.545 (asset criticality 0.5)

Context: The server accepts the TLS_RSA_WITH_3DES_EDE_CBC_SHA cipher suite, which uses the weak 3DES encryption vulnerable to birthday attacks (Sweet32) and lacks forward secrecy. An attacker within the network path can exploit this to decrypt HTTPS traffic, potentially exposing authentication credentials, personal data, or financial information. If the server's private key is later compromised, all previously recorded sessions using this cipher can be retroactively decrypted. This weakness may lead to data breaches, regulatory non-compliance (e.g., PCI DSS), and erosion of user trust.

Exploitability rationale: 3/3 models: Medium=3 (quorum 2 -> Medium)

Ensemble votes: deepseek-v4-pro=Medium, glm-5.2=Medium, kimi-k2.6=Medium

Remediation: Disable weak TLS cipher suites such as TLS_RSA_WITH_3DES_EDE_CBC_SHA by configuring Nginx to use only strong, modern cipher suites.

1. Locate the Nginx configuration file (typically /etc/nginx/nginx.conf or a virtual host file in /etc/nginx/sites-available/).
2. Modify the ssl_ciphers directive to specify a secure cipher string, e.g., 'EECDH+AESGCM:EDH+AESGCM:AES256+EECDH:AES256+EDH', ensuring weak ciphers like 3DES are excluded.
3. Optionally, set ssl_protocols to 'TLSv1.2 TLSv1.3' to disable outdated protocols.
4. Test the configuration for syntax errors with 'nginx -t'.
5. Reload Nginx to apply changes using 'systemctl reload nginx' or 'nginx -s reload'.
6. Verify the fix by scanning the service with a TLS testing tool (e.g., testssl.sh, nmap --script ssl-enum-ciphers) to confirm the weak cipher is no longer supported.

#8 Medium RDP Supports Weak Encryption – Remote Desktop service permits weaker encryption settings – Result: Security layer does not enforce strongest available mode

Host: 192.168.10.40:3389

CVSS: 5.6

Risk score: 0.53 (asset criticality 0.5)

Context: The Windows Server's RDP service on port 3389 permits weak encryption, failing to enforce the strongest available security layer. Real-world attacks include man-in-the-middle interception and session hijacking, where an attacker on the same network can downgrade the encryption to capture credentials or sensitive data. This exposes the organization to unauthorized access, data exfiltration, and lateral movement, potentially leading to full system compromise and regulatory non-compliance penalties.

Exploitability rationale: 2/3 models: Low=1, Medium=2 (quorum 2 -> Medium)

Ensemble votes: deepseek-v4-pro=Medium, glm-5.2=Medium, kimi-k2.6=Low

Remediation: Remediate by enforcing Network Level Authentication and high encryption with only strong TLS versions for RDP.

1. Open the Local Group Policy Editor (gpedit.msc) or Domain Group Policy Management Console.
2. Navigate to Computer Configuration\Administrative Templates\Windows Components\Remote Desktop Services\Remote Desktop Session Host\Security.
3. Set 'Set client connection encryption level' to Enabled and choose 'High Level'.
4. Set 'Require use of specific security layer for remote (RDP) connections' to Enabled and select 'SSL (TLS 1.0)'.
5. Set 'Require user authentication for remote connections by using Network Level Authentication' to Enabled.
6. Disable legacy TLS versions: open Registry Editor and under HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\SCHANNEL\Protocols, set Enabled=0 for TLS 1.0 and TLS 1.1 server keys; verify TLS 1.2 is Enabled=1.
7. Run 'gpupdate /force' and restart the Remote Desktop Services service (or reboot the server).

#9 Medium SSH Weak Key Exchange Algorithms Supported – Weak SSH algorithms detected – Result: diffie-hellman-group1-sha1 enabled

Host: 192.168.10.25:22

CVSS: 5.3

Risk score: 0.515 (asset criticality 0.5)

Context: The OpenSSH server on host 192.168.10.25 supports the weak key exchange algorithm diffie-hellman-group1-sha1, which uses a small 1024-bit modulus and SHA-1 hash, making it susceptible to cryptographic attacks like Logjam. An attacker with network access could capture SSH handshakes and decrypt past or current sessions, or actively force a downgrade to this algorithm and perform man-in-the-middle attacks to steal credentials or inject commands. This could result in unauthorized access to sensitive systems, data exfiltration, and lateral movement, potentially causing regulatory non-compliance, financial losses, and reputational damage.

Exploitability rationale: 3/3 models: Medium=3 (quorum 2 -> Medium)

Ensemble votes: deepseek-v4-pro=Medium, glm-5.2=Medium, kimi-k2.6=Medium

Remediation: Disable weak SSH key exchange algorithms like diffie-hellman-group1-sha1 by configuring the SSH server to use only strong, modern algorithms.

1. Edit the SSH server configuration file (typically /etc/ssh/sshd_config) with a text editor using root privileges.
2. Add or modify the 'KexAlgorithms' directive to exclude diffie-hellman-group1-sha1 and other weak algorithms; for example: KexAlgorithms curve25519-sha256,curve25519-sha256@libssh.org,diffie-hellman-group14-sha256,diffie-hellman-group16-sha512,diffie-hellman-group18-sha512,ecdh-sha2-nistp256,ecdh-sha2-nistp384,ecdh-sha2-nistp521
3. Restart the SSH service (e.g., sudo systemctl restart sshd or sudo service ssh restart) to apply the changes.
4. Verify that the weak algorithm is no longer offered using a tool like ssh-audit or nmap --script ssh2-enum-algos against the server.

#10 Medium SMTP Open Relay Check – Mail server configuration may permit unauthorized relaying – Result: Server accepted relay test pattern

Host: 192.168.10.50:25

CVSS: 4.0

Risk score: 0.45 (asset criticality 0.5)

Context: The Postfix mail server on 192.168.10.50:25 is configured as an open relay, allowing unauthenticated users to send arbitrary emails through it. Attackers can exploit this to distribute spam, phishing campaigns, or malware, using the organization's resources and reputation. This can lead to the server's IP address being blacklisted, disrupting legitimate email delivery and causing reputational damage. Additionally, the business may face legal liabilities, increased operational costs, and potential data breaches if sensitive information is intercepted.

Exploitability rationale: 2/3 models: High=1, Medium=2 (quorum 2 -> Medium)

Ensemble votes: deepseek-v4-pro=High, glm-5.2=Medium, kimi-k2.6=Medium

Remediation: Restrict mail relaying to trusted networks and authenticated users by configuring Postfix's `smtpd_relay_restrictions` and `mynetworks` parameters.

1. Back up the current Postfix configuration file (e.g., `/etc/postfix/main.cf`).
2. Add or update the `smtpd_relay_restrictions` directive: `smtpd_relay_restrictions = permit_mynetworks, permit_sasl_authenticated, reject_unauth_destination`
3. Ensure the `mynetworks` parameter includes only trusted IP ranges, such as `127.0.0.0/8`, `::1/128`, and internal management subnets (e.g., `mynetworks = 127.0.0.0/8, ::1/128, 192.168.10.0/24`).
4. Verify that any older `smtpd_recipient_restrictions` do not contain `permit_mynetworks` without a final `reject`; replace with the new `smtpd_relay_restrictions` if possible.
5. If remote users need to relay, configure SASL authentication (e.g., enable `smtpd_sasl_auth_enable = yes` and set `smtpd_sasl_security_options = noanonymous`).
6. Restart the Postfix service (e.g., `systemctl restart postfix` or `service postfix restart`).
7. Test the server with an open relay test tool or manually via telnet to ensure it no longer accepts relaying for unauthorized domains.

#11 Medium Directory Listing Enabled – The web server appears to allow directory listing – Result: Request to /backup/ returned indexed contents

Host: 192.168.10.60:8443

CVSS: 3.1

Risk score: 0.405 (asset criticality 0.5)

Context: Directory listing is enabled on the Apache Tomcat server for the /backup/ directory, exposing its contents to unauthenticated users. Attackers can browse and download potentially sensitive backup files, configuration data, or source code, which may contain credentials or other exploitable information. This could lead to serious data breaches, unauthorized access to internal systems, and non-compliance with data protection regulations. The business impact includes reputational damage, financial loss, and operational disruption due to compromised data.

Exploitability rationale: 2/3 models: Low=1, Medium=2 (quorum 2 -> Medium)

Ensemble votes: deepseek-v4-pro=Low, glm-5.2=Medium, kimi-k2.6=Medium

Remediation: Disable directory listing in Apache Tomcat by configuring the DefaultServlet to set 'listings' to 'false' to prevent exposure of directory contents.

1. Locate the Tomcat global web.xml file (usually \$CATALINA_HOME/conf/web.xml) or the specific web application's WEB-INF/web.xml.
2. Find the <servlet> entry for the 'default' servlet (org.apache.catalina.servlets.DefaultServlet) and ensure the 'listings' init parameter is set to 'false': <init-param><param-name>listings</param-name><param-value>>false</param-value></init-param>.
3. If a per-application configuration is needed, create or edit the WEB-INF/web.xml inside the deployed application folder (e.g., for the /backup context) to include the same default servlet configuration with listings disabled.
4. Remove any sensitive files from the /backup/ directory and consider placing an empty index.html file as an additional measure to prevent listings if the servlet configuration is bypassed.
5. Restart the Tomcat service to apply the changes.
6. Test the fix by accessing http://192.168.10.60/backup/ and verify that it no longer returns a directory listing (should respond with 403 Forbidden or 404 Not Found).

#12 Low TLS Certificate Expires Soon – TLS certificate is close to expiration – Result: Certificate expires within 14 days

Host: 192.168.10.25:443

CVSS: 4.3

Risk score: 0.345 (asset criticality 0.5)

Context: The TLS certificate on 192.168.10.25:443 (Linux Nginx) is expiring within 14 days, making the HTTPS service vulnerable to disruption. Real-world attack scenarios include man-in-the-middle attacks if users bypass browser warnings, or an attacker exploiting the expired certificate to spoof the service. If the certificate expires, legitimate users may be unable to access the application, leading to business interruption. This weakens security posture and erodes customer trust, especially if sensitive data is transmitted.

Exploitability rationale: 3/3 models: Low=3 (quorum 2 -> Low)

Ensemble votes: deepseek-v4-pro=Low, glm-5.2=Low, kimi-k2.6=Low

Remediation: Renew the TLS certificate before expiration to prevent service disruption and maintain secure encrypted connections.

1. Check the current certificate's expiration date and file path by examining Nginx configuration (e.g., 'nginx -T' or check .conf files for 'ssl_certificate' directive).
2. Obtain a renewed certificate from your Certificate Authority (CA) – for Let's Encrypt, run 'certbot renew' or an equivalent automated renewal command; for other CAs, follow their renewal process.
3. Replace the old certificate and private key files on the server with the new ones (usually in /etc/ssl/ or /etc/nginx/ssl/).
4. Ensure the Nginx configuration still points to the correct new certificate and key files; update paths if necessary.
5. Test the Nginx configuration for syntax errors with 'nginx -t'.
6. Gracefully reload Nginx to apply the new certificate: 'systemctl reload nginx' or 'nginx -s reload'.
7. Verify the new certificate is served by connecting to the host with a browser or 'openssl s_client -connect 192.168.10.25:443' and confirm the expiration date is updated.

#13 Low HTTP TRACE Method Enabled – The HTTP TRACE method is enabled – Result: TRACE / responded with 200 OK

Host: 192.168.10.25:80

CVSS: 3.7

Risk score: 0.315 (asset criticality 0.5)

Context: The Apache 2.4.52 web server on Ubuntu Linux has the HTTP TRACE method enabled, which causes the server to echo back received requests. In real-world attacks, this could be exploited via Cross-Site Tracing (XST) to steal cookies or sensitive headers if combined with cross-site scripting, though modern browsers restrict TRACE from JavaScript. The business impact includes potential session hijacking or information disclosure, but the risk is low due to browser mitigations and the low CVSS score of 3.7.

Exploitability rationale: 3/3 models: Low=3 (quorum 2 -> Low)

Ensemble votes: deepseek-v4-pro=Low, glm-5.2=Low, kimi-k2.6=Low

Remediation: Disabling the HTTP TRACE method prevents cross-site tracing attacks and limits information disclosure via the server's echo of request data.

1. Edit the Apache configuration file, typically `/etc/apache2/apache2.conf` or the site-specific virtual host file in `/etc/apache2/sites-available/`.
2. Add the directive `TraceEnable Off` in the appropriate server context, such as within the `<VirtualHost>` block or at the global level to apply to all sites.
3. Restart the Apache service using `'sudo systemctl restart apache2'` to apply the changes.
4. Confirm the fix by sending an HTTP TRACE request (e.g., `'curl -X TRACE http://192.168.10.25'`) and verifying the response is 405 Method Not Allowed or 403 Forbidden.

#14 Low HTTP Server Header Disclosure – Server version disclosed in HTTP headers – Result: Server: nginx/1.18.0

Host: 192.168.10.25:8080

CVSS: 2.6

Risk score: 0.26 (asset criticality 0.5)

Context: The vulnerability is the disclosure of the nginx server version (1.18.0) in HTTP response headers, which leaks software details to potential attackers. In real-world scenarios, attackers can leverage this information to fingerprint the server and identify known exploits or misconfigurations specific to this version, accelerating targeted attacks. While the direct impact is low (CVSS 2.6), this information gathering can assist in more critical attacks, such as exploiting unpatched vulnerabilities. Business impact includes increased risk of exploitation, potential non-compliance with security standards that require minimizing information exposure, and aiding in automated scanning and profiling of the organization's infrastructure.

Exploitability rationale: 3/3 models: Low=3 (quorum 2 -> Low)

Ensemble votes: deepseek-v4-pro=Low, glm-5.2=Low, kimi-k2.6=Low

Remediation: Disable Nginx server version disclosure by turning off server tokens to prevent attackers from identifying the server version.

1. Edit the main Nginx configuration file (default: /etc/nginx/nginx.conf) with a text editor.
2. Add or ensure the directive 'server_tokens off;' is present inside the 'http' block.
3. Run 'nginx -t' to validate the configuration syntax.
4. Reload Nginx using 'systemctl reload nginx' or 'nginx -s reload'.
5. Verify the Server header in HTTP responses now displays 'nginx' without a version number.

Ranked Summary

Rank	Score	Exploitability	Finding	Host
1	0.805	High	Apache HTTP Server < 2.4.58 Multiple Vulnerabilities – Outdated Apache version detected – Result: Banner returned: Apache/2.4.52 (Ubuntu)	192.168.10.25
2	0.76	High	Tomcat Outdated Version Detected – Application server version is outdated – Result: Banner indicates Apache Tomcat/9.0.30	192.168.10.60
3	0.69	Medium	SMB Signing Not Required – SMB signing is supported but not required – Result: Message signing: enabled but not required	192.168.10.40
4	0.65	High	MySQL Allows Empty Password Login – The service accepted authentication with a blank	192.168.10.30

Rank	Score	Exploitability	Finding	Host
			password – Result: Anonymous or blank-password login succeeded	
5	0.625	Medium	MySQL Server Unsupported Version – Database server appears to be running an unsupported version – Result: Version fingerprint indicates MySQL 5.6.x	192.168.10.30
6	0.555	Medium	Webmail Missing Security Headers – The webmail interface is missing recommended HTTP security headers – Result: Missing: Content-Security-Policy, X-Frame-Options	192.168.10.50
7	0.545	Medium	TLS Weak Cipher Suites Supported – Weak TLS cipher suites accepted – Result: TLS_RSA_WITH_3DES_EDE_CBC_SHA accepted	192.168.10.25
8	0.53	Medium	RDP Supports Weak Encryption – Remote Desktop service permits weaker encryption settings – Result: Security layer does not enforce strongest available mode	192.168.10.40
9	0.515	Medium	SSH Weak Key Exchange Algorithms Supported – Weak SSH algorithms detected – Result: diffie-hellman-group1-sha1 enabled	192.168.10.25
10	0.45	Medium	SMTP Open Relay Check – Mail server configuration may permit unauthorized relaying – Result: Server accepted relay test pattern	192.168.10.50
11	0.405	Medium	Directory Listing Enabled – The web server appears to allow directory listing – Result: Request to /backup/ returned indexed contents	192.168.10.60
12	0.345	Low	TLS Certificate Expires Soon – TLS certificate is close to expiration – Result: Certificate expires within 14 days	192.168.10.25
13	0.315	Low	HTTP TRACE Method Enabled – The HTTP TRACE method is enabled – Result: TRACE / responded with 200 OK	192.168.10.25
14	0.26	Low	HTTP Server Header Disclosure – Server version disclosed in HTTP headers – Result: Server: nginx/1.18.0	192.168.10.25

Risk score = (CVSS/10 × 0.5) + (Exploitability × 0.3) + (Asset × 0.2). Exploitability: High=1.0, Medium=0.5, Low=0.1.